

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Shaik Nuha	Reg. No.:	192472242
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I Shaik Nuha(192472242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk.Nuha

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions**3. Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. User Registration Validation System using Regular Expressions

Aim:

To develop a Python application that validates user credentials such as Email Address, Password, and Mobile Number using Regular Expressions before creating a user account.

Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

The application should validate:

- Email Address
- Password
- Mobile Number

using Regular Expressions.

Validation Rules:

Email

- Must start with an alphabet.
- May contain letters, digits, '.' and '_' before '@'.
- Domain should contain only alphabets.
- Valid extensions: .com, .org, .edu, .net, .in

Password

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@, #, \$, %, &, !)

Mobile Number

- Must contain exactly 10 digits.
- First digit should be between 6–9.

Requirements:

Software

- Python 3.x
- IDLE/VS Code

Module

- re (Regular Expressions)

Algorithm:

1. Import the `re` module.
2. Read Email, Password and Mobile Number.
3. Define Regular Expression patterns.
4. Validate Email.
5. Validate Password.
6. Validate Mobile Number.
7. Display validation results.

Python Program:

```
import re
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!])[A-Za-z\d@#%&!]{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
if re.match(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
if re.match(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Input

```
Enter Email: 192472242.simats@saveetha.com
Enter Password: Abhay@123
Enter Mobile Number: 94980309791
```

Sample Output

```
Invalid Email
Strong Password
Invalid Mobile Number
```

Explanation:

The program uses Python's Regular Expressions to compare user input against predefined validation rules. If the input satisfies the required pattern, it is considered valid; otherwise, it is marked invalid.

Result:

The User Registration Validation System successfully validated Email Address, Password, and Mobile Number according to the specified security policies.

2.Design a Finite State Automata (FSA) Simulator

Aim:

To develop a Python-based Deterministic Finite Automata (DFA) Simulator that accepts DFA components, simulates state transitions, and determines whether an input string is accepted or rejected.

Problem Statement:

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept DFA description
 - States
 - Input Alphabet
 - Transition Table
 - Initial State
 - Final State(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether each string is Accepted or Rejected.

Example:

Strings ending with "ab"

Requirements:

Software

- Python 3.x

Concepts Used

- Dictionary
- Loops
- Conditional Statements

Algorithm:

1. Define DFA states.
2. Define input alphabet.
3. Define transition table.
4. Define initial state.
5. Define final state.
6. Read number of input strings.

7. For each string:
 - Start from initial state.
 - Move according to transition table.
 - Store transition path.
8. If final state is reached, display Accepted.
9. Otherwise display Rejected.

Python Program:

```
transition = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}
initial_state = 'q0'
final_state = 'q2'
while True:
    string = input("\nEnter String (or type exit): ")
    if string.lower() == "exit":
        break
    state = initial_state
    path = [state]
    for ch in string:
        if ch not in ['a', 'b']:
            print("Invalid Input")
            break
        state = transition[state][ch]
        path.append(state)
    else:
        print("Transition Path:")
        print(" -> ".join(path))

        if state == final_state:
            print("Accepted")
        else:
            print("Rejected")
```

Sample Output

```
Enter String (or type exit): abab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q2
Accepted
```

```
Enter String (or type exit): |
```

Explanation:

The DFA simulator starts from the initial state and processes each character of the input string. Based on the transition table, it moves from one state to another until the string ends. If the final state reached is an accepting state, the string is accepted; otherwise, it is rejected.

Result:

The DFA Simulator successfully simulated state transitions and correctly identified accepted and rejected strings.

3. Smart Pattern Matching Engine using Regular Expressions

Aim:

To develop a Python-based Smart Pattern Matching Engine using Regular Expressions that searches different patterns such as dates, phone numbers, hashtags, mentions, prefixes, suffixes, and words.

Problem Statement:

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

- Word Search
- Prefix Search
- Suffix Search
- Date Search
- Phone Number Search
- Hashtag Search
- Mention (@username) Search

The application should display all matching patterns based on the user's menu selection.

Requirements:

Software

- Python 3.x

Module

- re (Regular Expressions)

Algorithm:

1. Import re module.
2. Store sample text.
3. Display menu options.
4. Accept user choice.
5. Perform selected search using Regular Expressions.
6. Display all matching patterns.
7. Repeat until user exits.

Python Program:

```
import re
text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

while True:
    print("\n----- MENU -----")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        dates = re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text)
        print("Dates:", dates)
    elif choice == 2:
        phones = re.findall(r'\b[6-9]\d{9}\b', text)
        print("Phone Numbers:", phones)
    elif choice == 3:
        hashtags = re.findall(r'#\w+', text)
        print("Hashtags:", hashtags)
    elif choice == 4:
        mentions = re.findall(r'@\w+', text)
        print("Mentions:", mentions)
    elif choice == 5:
        prefix = input("Enter Prefix: ")
        words = re.findall(r'\b' + prefix + r'\w*', text, re.IGNORECASE)
        print("Matching Words:", words)
    elif choice == 6:
        suffix = input("Enter Suffix: ")
        words = re.findall(r'\b\w*' + suffix + r'\b', text, re.IGNORECASE)
        print("Matching Words:", words)

    elif choice == 7:
        print("Program Ended.")
        break
    else:
        print("Invalid Choice")
```

Sample Output

```
----- MENU -----  
1. Search Date  
2. Search Phone Number  
3. Search Hashtag  
4. Search Mention  
5. Search Prefix  
6. Search Suffix  
7. Exit  
Enter your choice: 3  
Hashtags: ['#NLP']
```

Explanation

The program uses Python Regular Expressions to search for different patterns inside a text document. Based on the user's choice, it extracts dates, phone numbers, hashtags, mentions, prefixes, suffixes, or specific words and displays the matching results.

Result

The Smart Pattern Matching Engine successfully identified and displayed all matching patterns using Python Regular Expressions.